PDF Duplex Merge Tool - PLAN.md

Project Goal

Create a Python utility that merges two PDF files created from a simplex scanner workflow into a correctly ordered duplex PDF.

Typical Workflow

Original document:

Sheet 1: Front=Page 1 Back=Page 2 Sheet 2: Front=Page 3 Back=Page 4 Sheet 3: Front=Page 5 Back=Page 6

Scan Operation

Pass 1

Scan all fronts:

front.pdf Page 1 Page 3 Page 5

Pass 2

Flip stack and scan backs:
back.pdf Page 6 Page 4 Page 2

Desired Result

merged.pdf Page 1 Page 2 Page 3 Page 4 Page 5 Page 6

Phase 1 - Core Merge Engine

Inputs

front.pdf back.pdf

Processing

Step 1

Read all pages from

Step 2

Reverse back pages

Example:

6 4 2

becomes

2 4 6

Step 3

front[0] back[0] front[1] back[1] front[2] back[2]

Step 4

Generate
merged.pdf

Phase 2 - Odd Page Handling

Sometimes documents contain an odd number of pages.

Example

1 2 3 4 5

Front Scan:
1 3 5

Back Scan:
4 2

The program must detect:
front count = back count + 1

and merge correctly.

Result:

1 2 3 4 5

Phase 3 - Validation

Detect:

Missing Pages

Example:

front = 20 pages back = 15 pages

Warning:
Page counts do not match. Possible scanning error.

Empty PDFs

0 pages

Generate clear error.

Encrypted PDFs

Detect and report.

Phase 4 - Command Line Interface

```
python duplex_merge.py \
    front.pdf \
    back.pdf \
    output.pdf
```

Phase 5 – Logging

```
Front pages : 37
Back pages  : 36

Back pages reversed.

Merging...

Output:
tax_complete.pdf

Total Pages:
73

Success
```

Dry Run

```
--validate
```

Checks:

- PDF readable
- page counts
- merge feasibility

without creating output.

Preserve Metadata

Copy:

- Title
- Author
- Subject

to output PDF.

Phase 7 - GUI (Future)

Simple desktop app.

```
+---------------------------+
| Front PDF  [Browse]       |
| Back PDF   [Browse]       |
| Output PDF [Browse]       |
|                           |
| [ Merge ]                 |
+---------------------------+
```

Possible toolkits:

- Tkinter
- PySide6
- PyQt6

Technical Stack

Language

Python 3.12+

PDF Library

Preferred: pypdf

Reason:

- Active project
- Pure Python
- Good PDF support

- Easy page manipulation

Minimum Viable Product

Version 1.0 only needs:

☑Read front PDF
☑Read back PDF
☑Reverse back pages
☑nterleave pages
☑Handle odd page counts
☑Write merged PDF

Everything else can come later.

For this project, I'd actually start with less than 100 lines of Python and get the merge engine working first. Once that's proven with your tax documents, we can build the validation features and GUI on top of it.

Off page left